Experiment Reporting Framework

medium.com/airbnb-engineering/experiment-reporting-framework-4e3fcd29e6c0

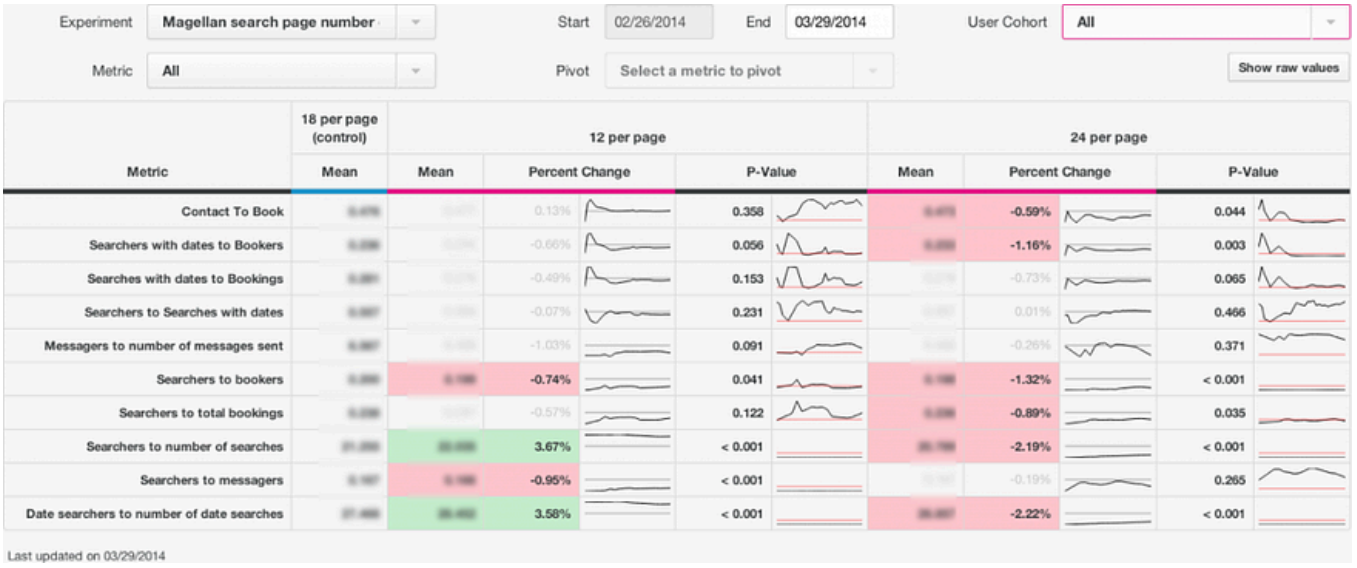
AirbnbEng May 29, 2014

Creative engineers and data scientists building a world where you can belong anywhere. http://airbnb.io

By Will Moss

At Airbnb we are always trying to learn more about our users and improve their experience on the site. Much of that learning and improvement comes through the deployment of controlled experiments. If you haven't already read our other post about experimentation I highly recommend you do it, but I will summarize the two main points: (1) running controlled experiments is the best way to learn about your users, and (2) there are a lot of pitfalls when running experiments. To that end, we built a tool to make running experiments easier by hiding all the pitfalls and automating the analytical heavy lifting.

Press enter or click to view image in full size



When designing this tool, making experiments simple to run was the primary focus. We also had some specific design goals that came out of what we've learned from running and analyzing experiments with our previous tool.

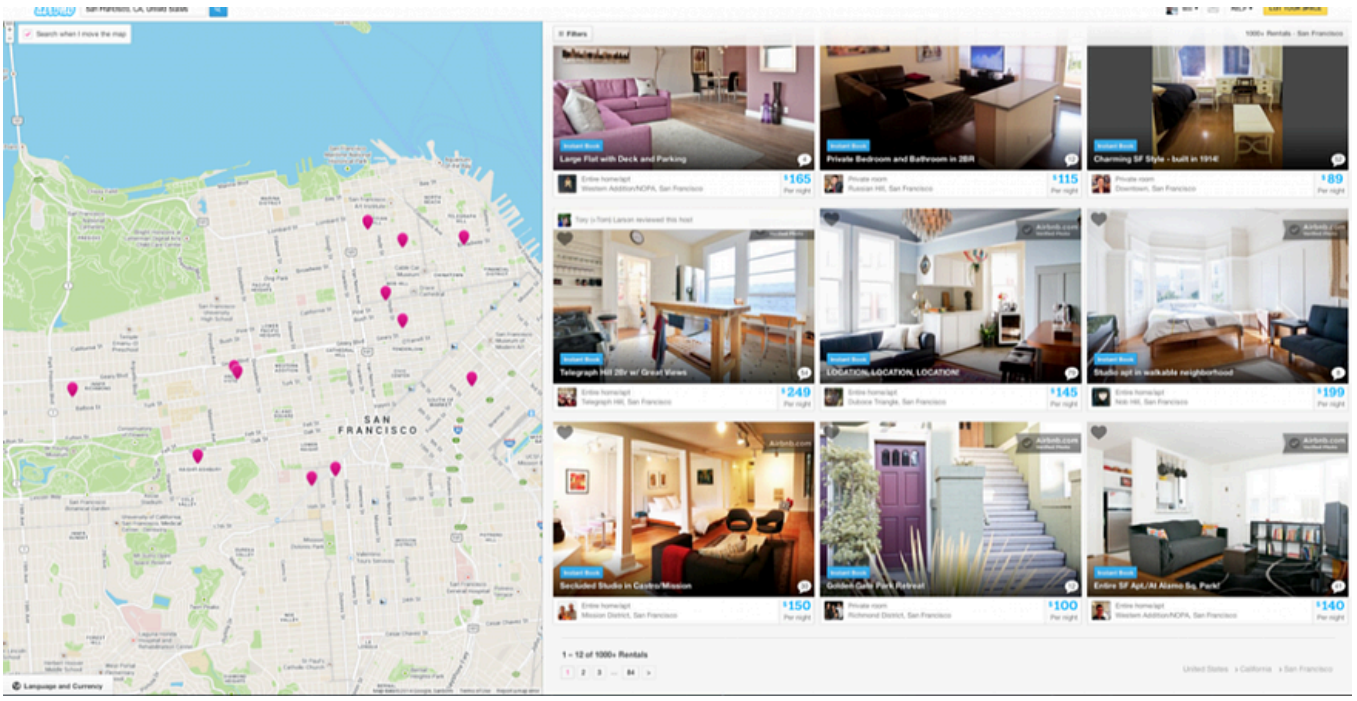
- Make sure the underlying data are correct. While this seems obvious, I'll show some examples below of problems we ran into before that caused us to lose confidence in our analysis.
- Limit the ways someone setting up an experiment could accidentally introduce bias and ensure that we automatically and reliably logged when a user was placed into a treatment for an experiment.
- Subject experimental changes to the same code review process we use for code changes–they can affect the site in the same way, after all.

- Run the analysis automatically so the barrier to entry to running (and learning from) an experiment is as low as possible.

Example Experiment

For the rest of this post, let's consider a sample experiment we might want to run and how we'd get there–from setting it up, to making the code changes, to seeing the results.

Press enter or click to view image in full size



Here is our current search results page, on the left we have a map of the results and on the right, images of the listings. By default, we show 18 results per page, but we wanted to understand how showing 12 or 24 results would affect user behaviour. Do users prefer getting more information at once? Or is it confusing to show too much? Let's walk through the process of running that experiment.

Declaring treatments

For declaring experiments we settled on yaml since it provides a nice balance between human and machine readability. To define an experiment, you need two key things–the subject and the treatments. The subject is who you want to run this experiment against. In this case, we choose visitor since not all users who search are logged in. If we were running an experiment on the booking flow (where users have to log in first) we could run the experiment against users. For a more in-depth look at the issues we've seen with visitor versus user experiments, check out our other post. Second, we have to define the treatments; in this case we have the control (of 18 results per page) and our two experimental groups, 12 and 24 results per page. The human_readable fields are what will be used in the UI.

Deploying

The next step is to implement this experiment in code. In the examples below, we'll be looking at Ruby code but we have a very similar function in Javascript that we can use for running experiments on cached pages.

The first argument is just the name of the experiment (from above). Then we've got an argument for each treatment above as well as a lambda function. The deliver_experiment function does three main things, (1) assign a user to a group (based on the specified subject), (2) log that the user was put into the treatment group, and (3) execute the provided lambda for the treatment group. You'll also notice one more argument, :unknown. This is there in the case we run into some unexpected failure. We want to make sure, even in the case that something goes horribly wrong, we still provide the user with a good experience. This group allows us to handle those cases by rendering that view to the user and logging that the unknown treatment was given (and, of course, also logging the error as needed).

Join Medium for free to get updates from this writer.

Remember me for faster sign in

This design may seem a little unorthodox, but there is a method behind the madness. To understand why we chose lambdas instead of something simpler like if statements, let's look at a few examples of doing it differently. Imagine, instead, we had a function that would return the treatment for a given user. We could then deploy an experiment like this:

This would work perfectly, and we could log which treatment a user was put into in the get_treatment function. What if, however, someone is looking at site performance later on and realizes that serving 24 results per page is causing the load times to skyrocket in China? They don't know about the experiment you're trying to run, but want to improve the user experience for Chinese users, so they come to the code and make the following change:

Now, what's happening? Well, we're still going to log that Chinese users are put into the 24 results per page group (since that happens on line 1) but, in fact, they will not be seeing 24 results per page because of the change. We've biased our experiment. While you could do that with the lambda too, we've found by making it very explicit that this code path is related to an experiment, people are more aware that they shouldn't be putting switching logic in there.

Let's look at another example, what about the following two statements?

In this case we have identical logic and the same users will see the treatment. The problem is that because the tests are short-circuited in the if statement, in the first case we correctly log only when a user actually sees the treatment. In the second case we have the same problem as above, where we log that Chinese users are seeing the 24 results per page treatment even though they are not.

Analyzing

Finally, once that's all done and deployed into the wild, we wait for the results to roll in. Currently we process the experiment results nightly, although it could easily be run more frequently. You can see a screenshot of the UI for the search results per page experiment in the image at the beginning of the post. At first glance, you'll see red and green cells. These cells signify metrics that we think are statistically significant based the methods presented in our previous post (red for bad and green for good). The uncolored cells with grey text represent metrics for which we are not yet sufficiently confident in the results. We also plot a spark line of the p-value and delta over time, which allows a user to look for convergence of these values.

As you can also see from the UI, we provide two other mechanisms for looking at the data, but I won't go into too much detail on those here. These allow for filtering the results, for example by new or returning users. We also support pivoting the results, so that a user could see how a specific metric performed on new vs. returning users.

Once we have significant results for the metrics we were interested in for an experiment, we can make a determination about the success (or failure) of that experiment and deploy a specific treatment in the code. To avoid building up confusing code paths, we try to tear down all completed experiments. Experiments can then be marked as retired, which will stop running the analysis, but retain the data so it can be still referred to in the future.

We plan to eventually open source much of this work. In the meantime, we hope this post gives you a taste of some of the decisions we made when designing this tool and why we made them. If you're trying to build something similar (or already have) we'd love to hear from you.

Check out all of our open source projects over at airbnb.io and follow us on Twitter

---

Originally published at nerds.airbnb.com on May 29, 2014.